

Webots - Reprojection de données

Compte Rendu de Travaux Pratiques

Formation : M2 SETI / Systèmes embarqués et traitement de l'information
Matière : C3 - Intelligence Artificielle pour la robotique

TP réalisé à l'UPSay - Université Paris-Saclay, le 16 janvier 2026.

Table des matières

1 Télémètre laser embarqué	3
1.1 Récupération des données	3
1.2 Données sous forme xy	3
2 Télémètre laser et localisation	6
2.1 Biais d'estimation sur la projection	6
2.2 Correction de la localisation par méthode ICPSVD	6
2.3 Fonction ICP	8
2.4 Correction de la position du robot	10
2.5 Validation des résultats	11
2.6 Méthode sans odométrie	15
Conclusion	17

Chapitre 1

Télémètre laser embarqué

L'objectif du Lidar embarqué sur le robot est de recréer un nuage de points autour de lui pour évaluer son terrain. L'objectif est de convertir des distances en points cartésiens projetés dans le repère du monde Webots.

1.1 Récupération des données

Afin de récupérer les données lidar du robot thymio, j'emploie une bibliothèque dédiée :

```
1  # LIDAR
2
3  lidar = robot.getDevice('lidar')
4  lidar.enable(int(robot.getBasicTimeStep()))
5  lidar.enablePointCloud()
6  point_cloud = lidar.getRangeImage()
7  fov = lidar.getFov()
8  angle_lidar = fov/2
9  angle_increment = fov / lidar.getHorizontalResolution()
```

Listing 1 – API proposée par la bibliothèque

1.2 Données sous forme xy

La principale difficulté rencontrée dans ce TP est la mise sous forme de données cartésiennes. En effet, j'ai remarqué que la fonction de transformée de repère ne me permettait pas d'obtenir une projection dans le repère global correcte.

Pour corriger ce point, j'ai réalisé une conversion polaire -> cartésien en projetant la distance du point i en utilisant la position actuelle du robot et son orientation θ .

- $P_x = x_{rob} + \text{distance} \cdot \cos(\theta + \alpha_{scan})$
- $P_y = y_{rob} + \text{distance} \cdot \sin(\theta + \alpha_{scan})$

Cette transformée ne permet pas une projection dans un plan 3D mais seulement 2D (Thymio n'étant pas sujet à un déplacement sur l'axe Z dans ce TP).

Un autre problème rencontré est l'effet miroir posé par un ordre de lecture des données inversé (gauche/droite). Pour cela je décrémente l'angle de balayage plutôt que de l'incrémenter.

```
1
2  #point de départ
3  angle_lidar = fov/2
4  for i in point_cloud:
5
6      #filtrage valeur infinie
7      if not math.isinf(i):
8          #rotation robot + angle rayon i
9          global_angle = theta + angle_lidar
10
11         #projection de x
12         p_x = x + i * math.cos(global_angle)
13
14         #projection de y
15         p_y = y + i * math.sin(global_angle)
16
17         #stockage des données
18         lidar_points_x.append(p_x)
19         lidar_points_y.append(p_y)
20
21         #sens de balayage des données
22         angle_lidar -= angle_increment
23
24
25  #affichage graphique :
26  plt.plot(lidar_points_x, lidar_points_y, 'b.', markersize=2, label='Lidar')
```

Listing 2 – Mise en forme des données au format xy

En observant le changement de base, je peux vérifier la cohérence des résultats :

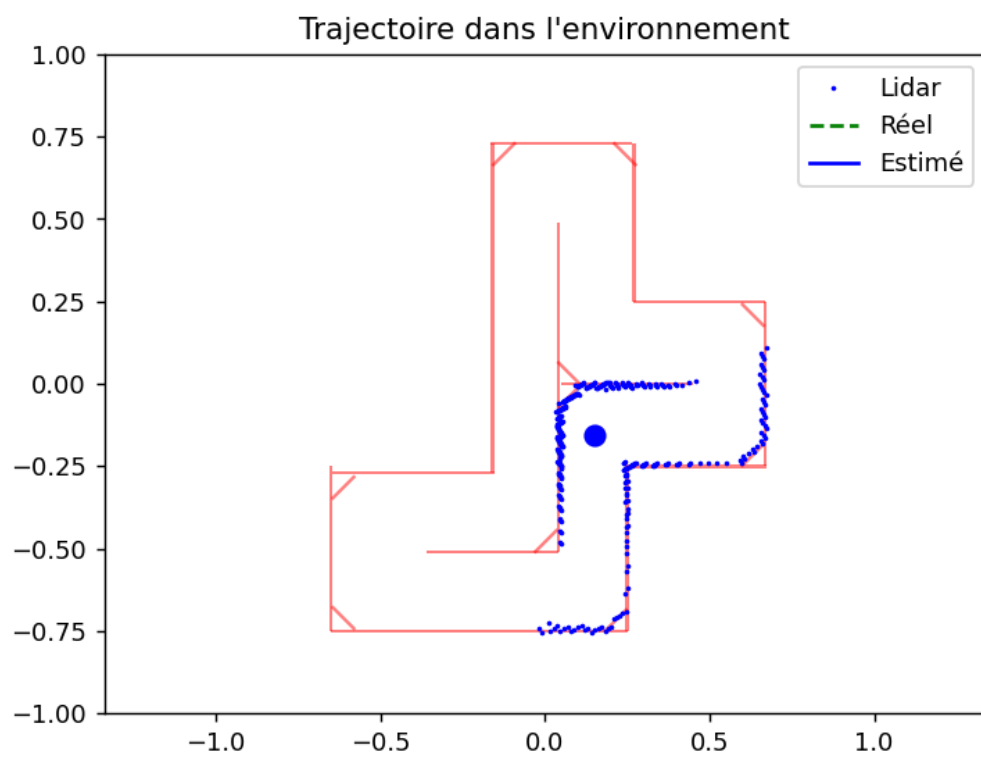


Figure 1.1 – Affichage du nuage de points du LiDAR

Chapitre 2

Télémètre laser et localisation

2.1 Biais d'estimation sur la projection

En reprenant le code du TP précédent pour faire contrôler thymio dans le labyrinthe, je remarque que le nuage de points perçu se décale autant que l'erreur d'estimation de position : le repère du lidar est le même que thymio.

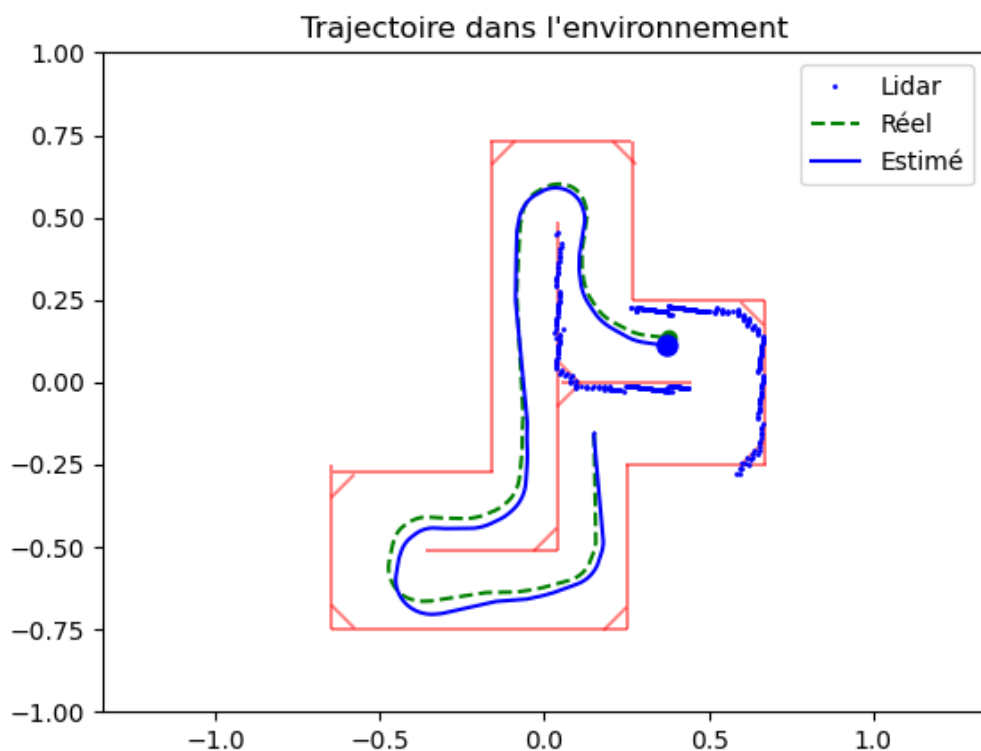


Figure 2.1 – Observation d'un biais dans la projection du télémètre laser sur les murs

2.2 Correction de la localisation par méthode ICPSVD

L'objectif de cette partie est de corriger l'erreur d'estimation en utilisant un algorithme ICP.

Pour cela, il est proposé de discrétiser les murs du labyrinthe en une série de points espacés d'1cm pour pouvoir utiliser les algorithmes donnés dans le sujet.

Pour réaliser cette discrétisation, j'utilise la fonction suivante, qui récupère les données extraites des murs déjà employées dans le TP2 :

```
1 def get_walls_point_cloud(walls_config, step=0.01): #step de 1cm.
2     fixed_x = []
3     fixed_y = []
4
5     for wall in walls_config:
6         cx, cy = wall['pos']
7         w, h = wall['size']
8         angle = wall['angle']
9
10        #si largeur > hauteur -> itération sur x sinon y
11        if w > h:
12            #création d'un tableau de points de -w/2 à w/2
13            lengths = np.arange(-w/2, w/2, step)
14            local_x = lengths
15            local_y = np.zeros_like(lengths) #y est au milieu
16        else:
17            lengths = np.arange(-h/2, h/2, step)
18            local_x = np.zeros_like(lengths) #x est au milieu
19            local_y = lengths
20
21        #rotation des points
22        c, s = np.cos(angle), np.sin(angle)
23
24        #rotation 2D
25        rot_x = local_x * c - local_y * s
26        rot_y = local_x * s + local_y * c
27
28        #translation et stockage
29        fixed_x.extend(rot_x + cx)
30        fixed_y.extend(rot_y + cy)
31
32    return fixed_x, fixed_y
```

Listing 3 – Génération des points des murs discrétisés

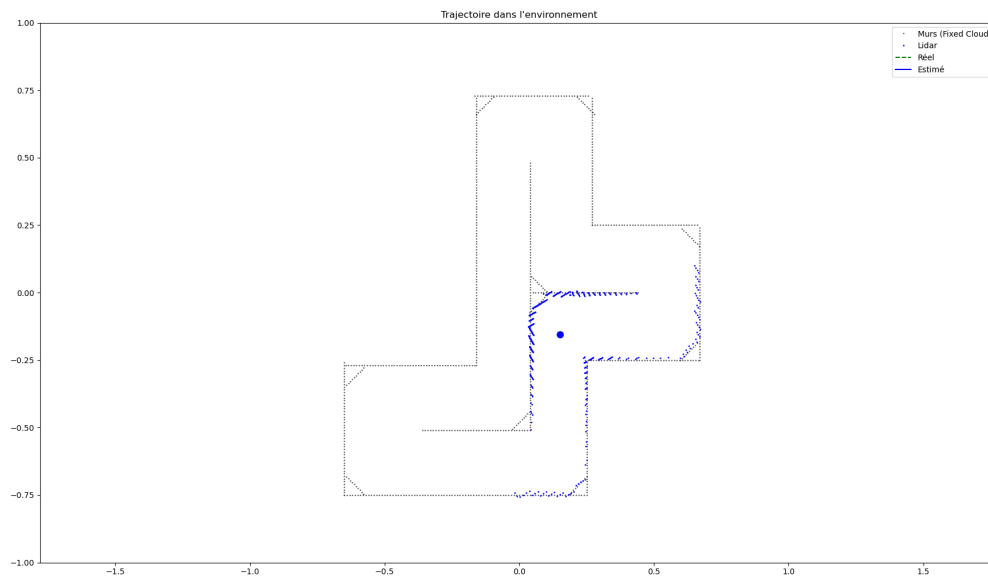


Figure 2.2 – Visualisation des points fixes présents sur les murs avec espacement de 1 cm

2.3 Fonction ICP

La fonction ICP permet de projeter les points du nuage sur les murs afin de calculer une transformée de repère qu'on appliquera au robot pour corriger sa position. Voici son implémentation commentée en python :

```

1
2 iterative closest point avec singular value decomposition
3 def ICPSVD(fixedX,fixedY,movingX,movingY):
4     #https://fr.wikipedia.org/wiki/Iterative_Closest_Point
5     #d'après wikipédia
6     reqR = np.identity(3) #matrice identité de rotation
7     reqT = [0.0, 0.0, 0.0] #matrice nulle de translation
8     fixedt = []
9     movingt = []
10
11     #nuage fixe
12     for i in range(len(fixedX)):
13         fixedt.append([fixedX[i], fixedY[i], 0])
14
15     #nuage calculé
16     for i in range(len(movingX)):
17         movingt.append([movingX[i], movingY[i], 0])
18
19     #1 - sélection des points dans les nuages de départ
20     moving = np.asarray(movingt)
21     fixed = np.asarray(fixedt)
22
23     #2- mise en correspondance (voisin proche)
24     n = np.size(moving,0)
25     TREE = KDTree(fixed)
26
27     #mise en correspondance
28     for i in range(10):
29         #3- pondération des paires de points
30         distance, index = TREE.query(moving)
31         #4 - rejet (absent ici, tous les points sont comptés)
32         #5 - critère de distance (erreur quadratique)
33         err = np.mean(distance**2)
34
35         #centroïdes
36         com = np.mean(moving,0) #nuage calculé
37         cof = indxtMean(index,fixed) #nuage fixe
38
39         #6 - Minimisation du critère de distance
40         W = np.dot(np.transpose(moving),indxtfixed(index,fixed)) - n*np.outer(com,cof)
41         #valeur singulières SVD
42         #https://en.wikipedia.org/wiki/Kabsch_algorithm
43         # -> calculer la matrice de rotation optimale.
44         U , _ , V = np.linalg.svd(W, full_matrices = False)
45         tempR = np.dot(V.T,U.T) #rotation locale
46         tempT = cof - np.dot(tempR,com) #translation locale (centroïde fixe - rotation * centroïde calculé)
47
48         #maj avec la transformée
49         moving = (tempR.dot(moving.T)).T #rotation
50         moving = np.add(moving,tempT) #translation
51         reqR=np.dot(tempR,reqR)
52         reqT = np.add(np.dot(tempR,reqT),tempT)
53
54     return reqR, reqT, moving
55

```

Listing 4 – Commentaire sur la fonction ICP

Enfin, voici comment cette fonction est appelée dans mon code :

```

1
2 reqR, reqT, moving_corrected = ICPSVD(wall_points_x, wall_points_y, lidar_points_x, lidar_points_y)
3 icp_x = moving_corrected[:,0]
4 icp_y = moving_corrected[:,1]
5 plt.plot(icp_x, icp_y, 'm.', markersize=2, label='Lidar corrigé ICP')

```

Listing 5 – Appel à la fonction ICPSVD

Cela nous permet d'obtenir le nuage de points rouge représentant la projection des informations du LiDAR sur les murs :

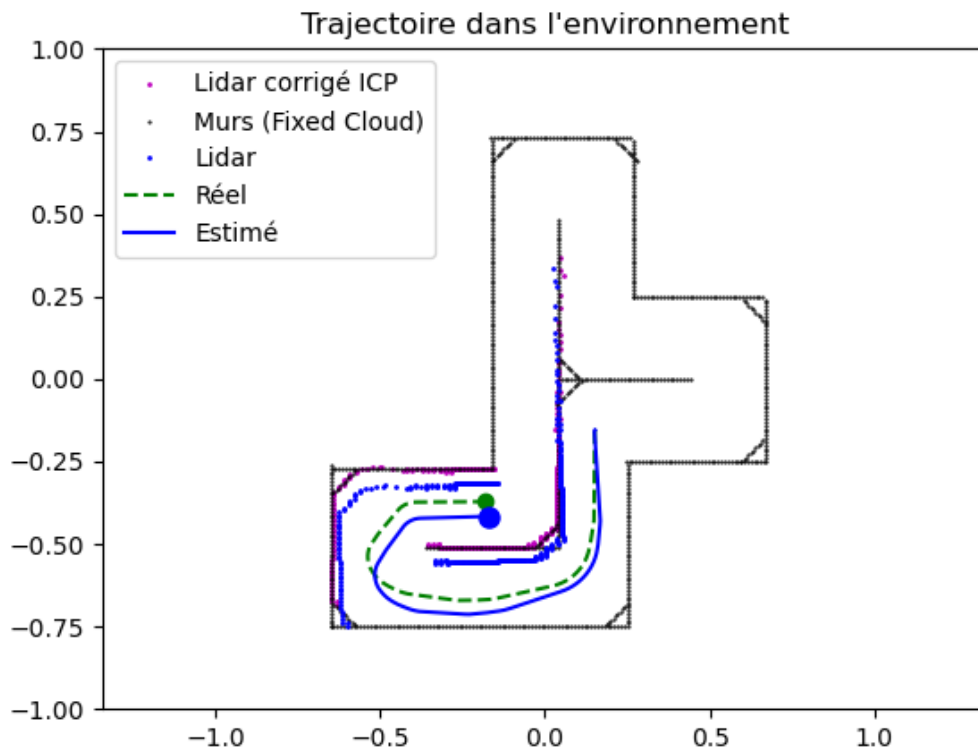


Figure 2.3 – Correction de la projection des points du Lidar sur les murs

On peut voir ici que la correction de la projection des points par l'algorithme donné suit bien les murs du parcours. On peut donc vérifier ici la cohérence du correctif par rapport à la projection biaisée du lidar (en bleu sur l'image).

2.4 Correction de la position du robot

Pour corriger la position du robot, on va lui appliquer sur sa position, la même transformée de repère que celle appliquée par l'algorithme ICP. Une fois la matrice de rotation R et le vecteur de translation T obtenus grâce à l'algorithme ICP, il est nécessaire de corriger l'état du robot (x, y, θ) .

Une première approche consistait à corriger l'orientation θ en ajoutant l'angle de rotation, et à corriger la position (x, y) en ajoutant simplement le vecteur de translation T . Cependant, cette méthode ne fonctionnait qu'à moitié : le robot ne suivait pas la même transformée de repère que la projection des nuages de points.

Le problème venait du fait que l'ICP calcule une transformation rigide globale. La matrice de rotation R n'affecte pas seulement l'orientation du robot (θ), mais également ses coordonnées (x, y) dans le repère global. En oubliant l'effet de la rotation sur le vecteur position, le robot se retrouvait corrigé d'une mauvaise manière.

Pour résoudre ce problème, j'ai traité la position estimée du robot comme un vecteur $P_{est} = [x, y, 0]^T$ auquel j'ai appliqué la transformation complète :

$$P_{corr} = R \cdot P_{est} + T \quad (2.1)$$

Voici l'implémentation permettant de mettre à jour correctement la pose du robot :

```

1
2  #attente de 5 secondes.
3  if current_sim_time - last_icp_compute_time >= icp_interval:
4
5      print(f"\n--- Recalage ICP (t={current_sim_time:.2f}s) ---")
6
7      #calcul ICP
8      reqR, reqT, moving_corrected = ICPSVD(wall_points_x, wall_points_y, lidar_points_x, lidar_points_y)
9
10     #nuage corrigé
11     icp_x = moving_corrected[:, 0]
12     icp_y = moving_corrected[:, 1]
13
14     #correction
15     old_x, old_y = x, y
16
17     #position au format vecteur
18     robot_pos_vector = np.array([x, y, 0.0])
19
20     #reqR matrice de rotation
21     #reqT vecteur de translation
22     corrected_robot_pos = np.dot(reqR, robot_pos_vector) + np.array(reqT)
23
24     #coordonnées corrigées
25     x = corrected_robot_pos[0]
26     y = corrected_robot_pos[1]
27
28     #mis à jour de theta.
29     delta_theta_icp = math.atan2(reqR[1, 0], reqR[0, 0])
30     theta += delta_theta_icp
31     theta = math.atan2(math.sin(theta), math.cos(theta))
32
33     last_icp_compute_time = current_sim_time

```

Listing 6 – Correction de la pose du robot par transformation affine

Cette correction permet d'assurer que la position du robot reste cohérente avec le recalage du nuage de points lidar observé précédemment.

2.5 Validation des résultats

Afin de valider le recalage du robot par l'algorithme ICP, je mets le robot en défaut et j'observe comment évolue sa position corrigée.

Ici, je place le robot face à un mur et le fais avancer. Ainsi on peut voir sa trajectoire estimée aller au delà du mur. L'odométrie est donc faussée.

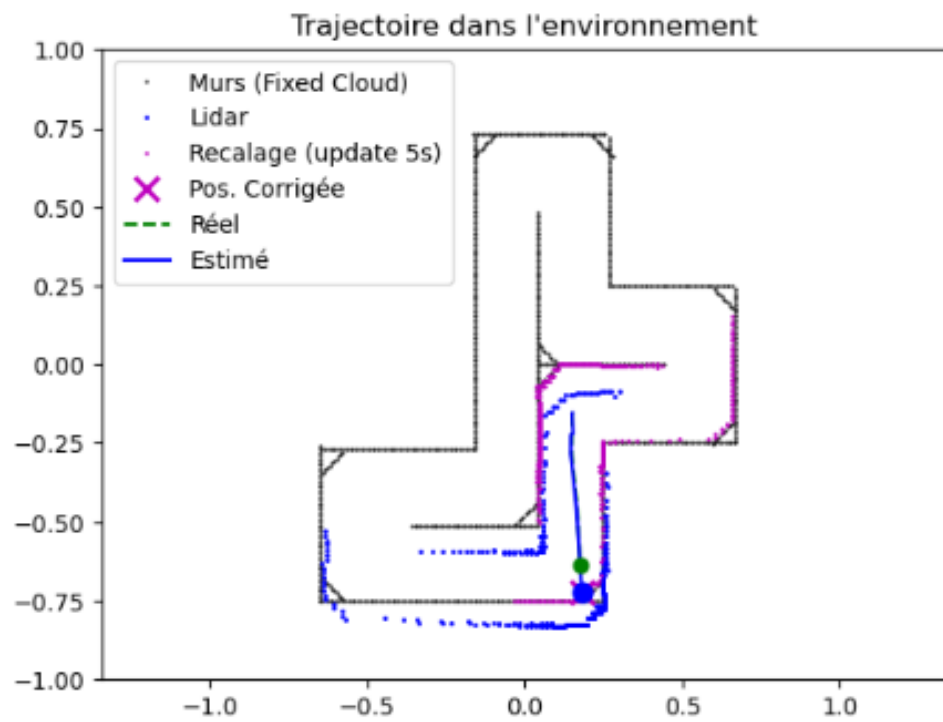


Figure 2.4 – Position faussée par une erreur d'estimation de l'odométrie

Au terme des cinq secondes, le programme replace la position estimée du robot à sa position réelle avec l'algorithme ICP.

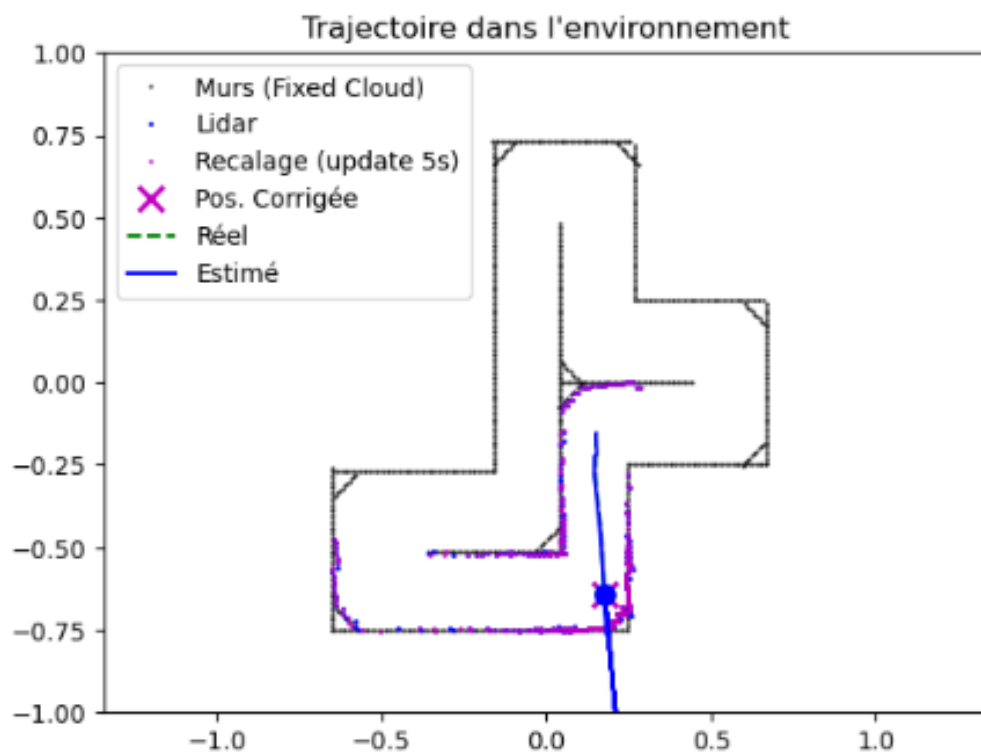


Figure 2.5 – Position corrigée au bout de cinq secondes

Lorsque le robot repart, sa nouvelle trajectoire reprend à partir de la position corrigée :

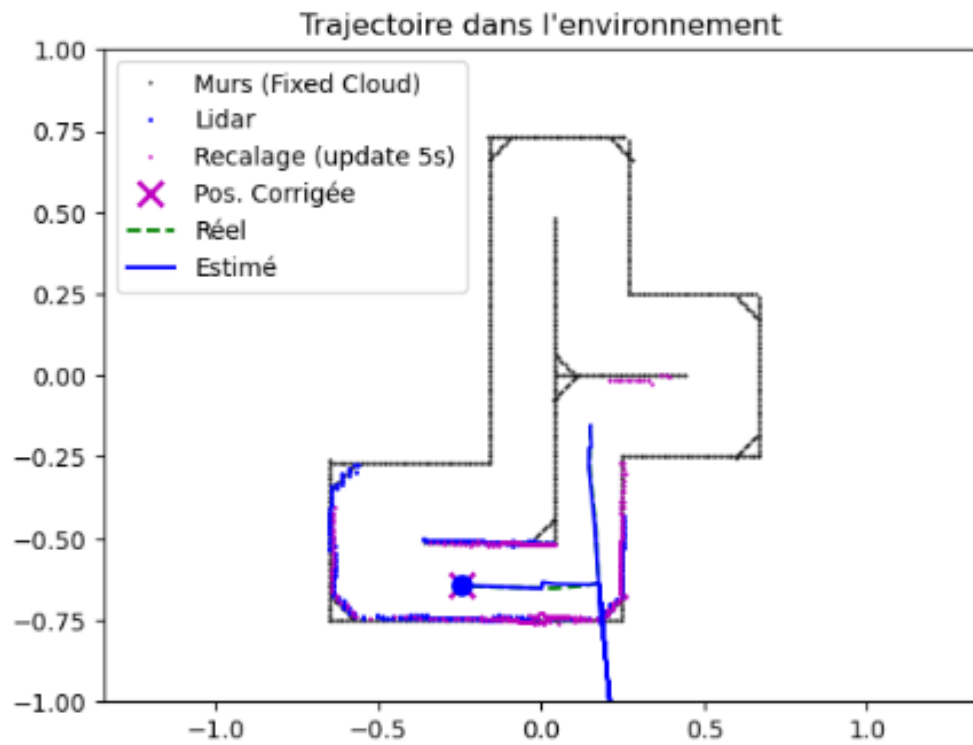


Figure 2.6 – La trajectoire estimée reprend à partir de la position corrigée

Cela permet de démontrer que l'algorithme permet de revenir d'une situation faussée et de corriger correctement l'état estimé du robot.

Pour prouver graphiquement que la solution avec recalage est meilleure que la solution sans, j'ai ajouté une liste qui mesure à chaque itération l'erreur de position estimée par rapport à la position réelle : $\sqrt{(x_{est} - x_{rel})^2 + (y_{est} - y_{rel})^2}$. Le graphique suivant montre l'erreur de position en fonction du temps avec les deux méthodes :

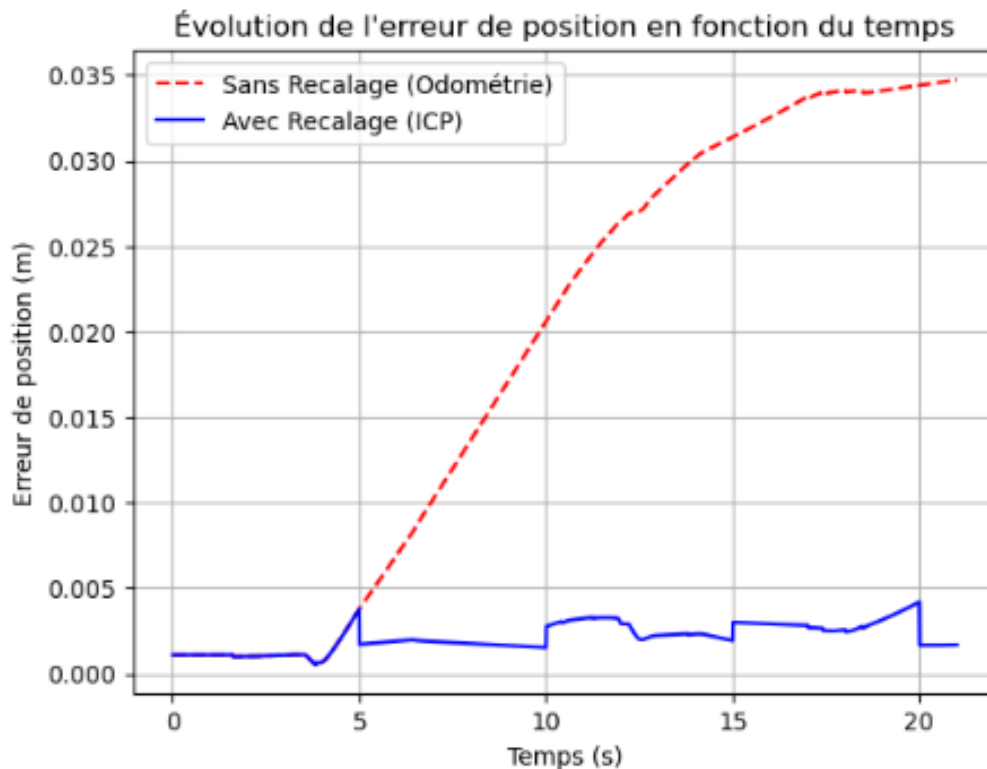


Figure 2.7 – Evolution de l’erreur de position en fonction du temps

Comme le montre la courbe rouge, l’erreur augmente progressivement avec le temps. Chaque nouvelle erreur d’odométrie s’accumule. Sans correction, le robot fini par croire à une position sur laquelle il n’est pas.

La courbe bleue montre une erreur bornée qui est corrigée toutes les cinq secondes. l’erreur s’accumule entre deux calculs due à l’odométrie.

Le recalage est donc indispensable dans cet exercice pour assurer une navigation longue durée. Cependant, j’ajoute que dans ce programme, on prend la correction comme étant la vérité mais nous pourrions intégrer cette affirmation dans un filtre de kalman étendu (EKF) pour pondérer l’odométrie et la méthode ICP. J’avais déjà auparavant réalisé un projet d’estimation de position par IMU et GNSS qui reprenait quelques bases de cette problématique.

2.6 Méthode sans odométrie

Pour réaliser l’estimation de trajectoire sans odométrie, je réduis l’intervalle de cinq à une demi-seconde et je ne recalcule plus les nouvelles positions qu’avec la méthode ICP.

Enfin, une fois les nouvelles coordonnées calculées, j’obtiens ces mesures d’erreur qui me permettent de comparer les trois méthodes :

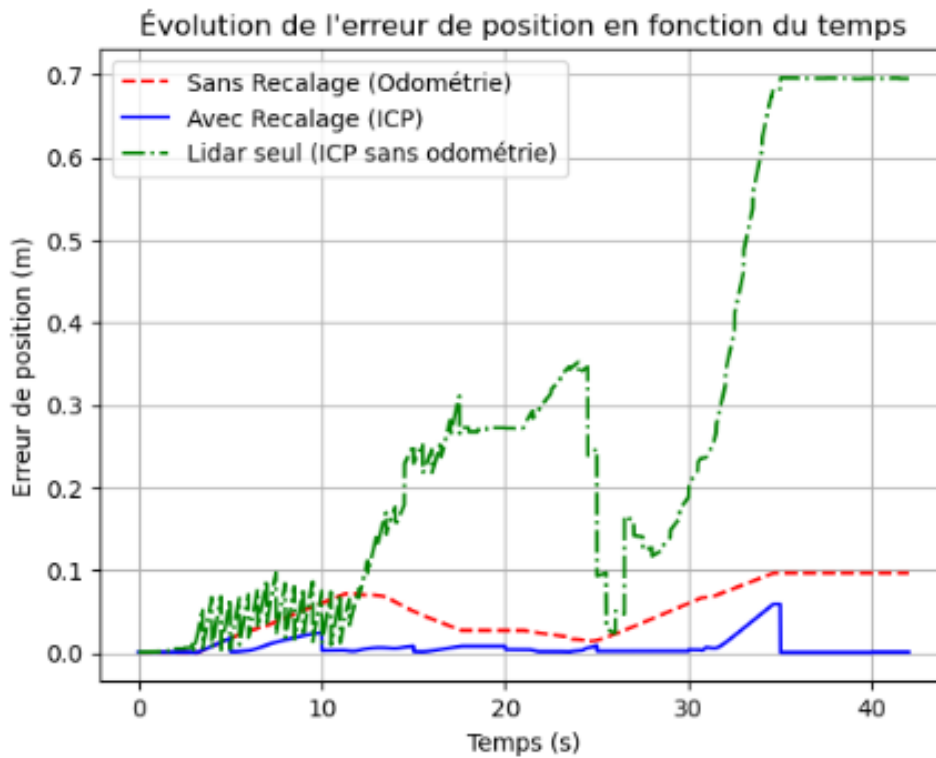


Figure 2.8 – Evolution des estimations en fonction de la méthode employée dans le temps

Je remarque ici que l'erreur d'estimation de la méthode avec lidar seul oscille dans un premier temps assez fortement et fini par augmenter de manière très forte. On remarque cependant une correction proche de la valeur correcte autour des 25 secondes de la simulation. J'émet l'hypothèse le fait que cette méthode n'exploite pas la notion du mouvement et qu'elle peut devenir aveugle par le manque de points de repère dans certains passages du labyrinthe. Il n'y a donc pas suffisamment de richesse d'information géométrique pour recalculer la position du robot.

Ainsi, la méthode de fusion de données entre odométrie et ICP me semble la plus prometteuse des trois méthodes.

Conclusion

Ce TP consacré à la télémétrie laser m'a permis d'aborder une problématique fondamentale en robotique : la localisation précise d'un robot en environnement connu malgré les erreurs des capteurs.

L'utilisation de l'algorithme ICP couplé à l'odométrie vue lors du premier TP apporte une solution au problème d'accumulation d'erreurs dans la navigation du robot. Cette méthode souffre d'un manque de robustesse dans un environnement trop propre mais une fusion avec l'odométrie s'impose comme une solution optimale pour de la localisation.